## 第7章 查找

7.1 静态查找表

7.2 动态查找表

7.3 哈希表

## 查找的概念

□查找/检索：根据给定的K值，在查找表中确定一个关键字等于给定值K的记录或数据元素。

➢查找的对象是查找表，采用何种查找方法，首先取决于查找表的组织。

➢查找表是记录的集合，而集合中的元素之间是一种完全松散的关系，因此，查找表是一种非常灵活的数据结构，可以用多种方式来存储。

## 查找的两种基本形式

➢静态查找(Static Search)

在查找时只对数据元素进行查询或检索，查找表称为静态查找表。

➢ 动态查找(Dynamic Search)

在实施查找的同时，插入查找表中不存在的记录，或从查找表中删除已存在的某个记录，查找表称为动态查找表。

## 查找方法评价指标

查找过程中主要操作是关键字的比较，查找过程中关键字的平均比较次数(平均查找长度ASL：Average Search Length)作为衡量一个查找算法效率高低的标准。

对n个记录进行查找，查找成功的平均查找长度为

$$ASL = \sum_{i=1}^{n} P_i \cdot C_i$$

其中，Ci是查找第i个记录所需的比较次数；Pi是查找第i个记录的查找概率，且∑Pi＝1。若每个记录的查找概率均等，即Pi＝1/n，则

$$ASL = \frac{1}{n}\sum_{i=1}^{n} C_i$$

## 7.1 静态查找表

7.1.1 顺序表的查找

7.1.2 有序表的查找

7.1.3 索引顺序表的查找

## 7.1.1顺序表的查找——顺序查找

➢基本思想：从查找表的一端开始，顺序扫描查找表，依次将扫描到的结点关键字和给定值k相比较，若当前扫描到的结点关键字与k相等，则查找成功；若扫描结束后，仍未找到关键字等于k的结点，则查找失败。

例:序列(5， 46， 70 ， 94 ，42， 13 ， 55， 17)
分析：查找k＝55的关键字，则查找成功，需比较7次；
查找k＝12的关键字，则查找失败。

➢算法分析：

$$ASL = \frac{1}{n}\sum_{i=1}^{n} C_i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2}$$

## 7.1.2 有序表的查找——折半查找

要求查找表中的结点必须按关键字值的递增/递减顺序排列。

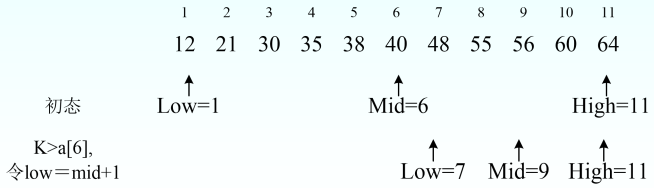
➢基本思想：首先用要查找的关键字k与中间位置的结点的关键字相比较，这个中间结点把查找表分成两个子表，若比较结果相等则查找成功；若不相等，再根据k与该中间结点关键字的比较大小确定下一步查找哪个子表，这样递归进行下去，直到找到满足条件的结点或该查找表中没有这样的结点。

## 对分查找的例题

例：在11个数据元素的有序表(12, 21, 30, 35, 38, 40, 48, 55, 56, 60, 64)中，分别查找k为56和36的数据元素。

分析：指针*low*和*high*分别指示待查元素所在范围的下界、上界，指针 $mid = \lfloor (low + high)/2 \rfloor$ 指示区间的中间位置。

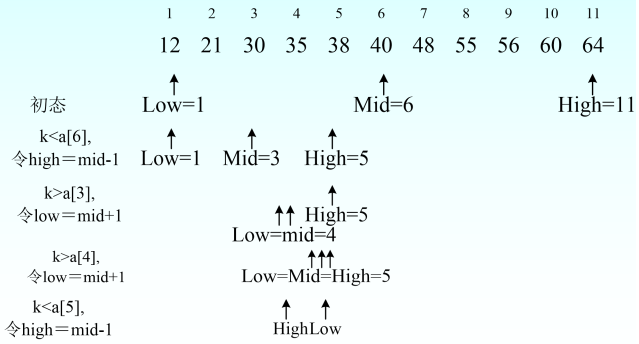
解：（1）k＝56




查找成功！！

## 对分查找的例题（续）

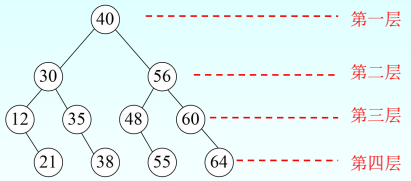
（2）k＝36




High<low　　查找失败！！

## 对分查找的算法分析

有序表(12, 21, 30, 35, 38, 40, 48, 55, 56, 60, 64)的查找过程




➢算法分析：对分查找恰好是一条从判定树的根到被查结点的一条路径，而比较的次数恰好是树的深度。假设表长为n，树深$h=\log_2(n+1)$，第k层上结点的个数为$2^{k-1}$，则

$$ASL = \frac{1}{n}\sum_{i=1}^{n} C_i = \frac{1}{n}\sum_{i=1}^{h} i \cdot 2^{i-1} = \frac{n+1}{n}\log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

## 对分查找的思考题

如果将 $mid = \lfloor (low + high)/2 \rfloor$ 改成 $mid = \lceil (low + high)/2 \rceil$，画出有序表(12, 21, 30, 35, 38, 40, 48, 55, 56, 60, 64)的查找判定树。

## 7.1.3索引顺序表的查找——分块查找

•基本思想：

把查找表分成若干块，每一块中的元素存储顺序是任意的，但块与块之间必须按关键字大小有序排列，即前一块中的最大关键字小于后一块中的最小关键字，将这些块的最大关键字构成一个索引表。

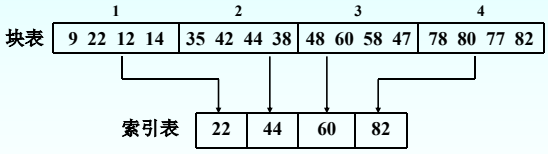
•分块查找步骤：

1）查索引表，确定待查找的结点属于哪一块（顺序、对分查找均可）

2）查块表，在所定块中查找要查的结点（顺序查找）

## 分块查找的例题

例：对长度为16，关键字为(9,22,12,14,35,42,44,38,48,60,58,47,78,80,77,82)的线性表，将其分为4块，试画出其块表及索引表结构。




## 分块查找的算法分析

➢算法分析：分块查找实际上进行两次查找，则整个算法的平均查找长度是两次查找的平均查找长度之和。
假设有n个结点，分成b块，每块有s=n/b个结点，每块的查找结点概率为1/b，块内每个结点的查找概率为1/s，则

$$ASL \approx \log_2(b+1)-1+(s+1)/2 = \log_2(\frac{n}{s}+1)+\frac{s}{2}$$

索引表（对分查找）　块表（顺序查找）

$$ASL=(b+1)/2+(s+1)/2=(b+s)/2+1$$

索引表（顺序查找）　块表（顺序查找）

## 分块查找的例题

例：采用分块查找时，若线性表中共有2500个元素，查找每个元素的概率相同，假设采用顺序查找来确定元素所在的块时，每个块应分（　）个元素最佳。

分析：$ASL=(b+s)/2+1=(b+n/b)/2+1$

其中，对于b+ n/b，由均值不等式知，当b= n/b时有最小值，此时 $b=\sqrt{n}$ 。则最理想块长为 $\sqrt{2500}=50$。

## 7.2 动态查找表

### 7.2.1 二叉排序树

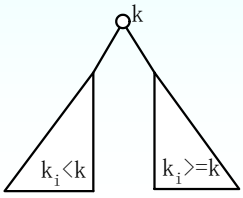
### 7.2.2 平衡二叉树

## 7.2.1二叉排序树

二叉排序树的定义：

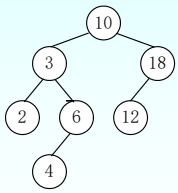
二叉排序树或是空树，或是一棵具有如下性质的二叉树：

1) 若它的左子树非空，则左子树中的所有结点的键值都小于根结点的键值；
2) 若它的右子树非空，则右子树中的所有结点的键值都不小于（大于或等于）根结点的键值；
3) 它的左、右子树本身又各是一棵二叉排序树。




## 二叉排序树的例题

例：判断下列的二叉树是否是一棵排序树，并按中序遍历这个树。




若按中序遍历该树，则得到一个排好顺序的结点序列：

{2，3，4，6，10，12，18}

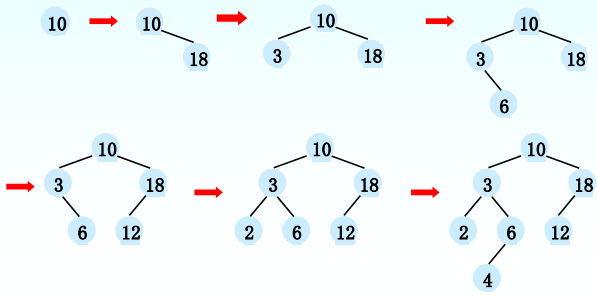
结论：利用二叉排序树可将一无序的序列转换成一有序序列。

## 二叉排序树的建立（插入）

原则：

保证插入或删除结点后的二叉树仍符合二叉排序树的特点

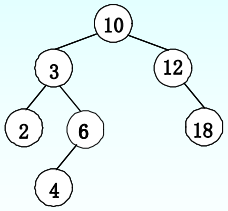
例：将序列{10, 18, 3, 6, 12, 2, 4}构成一棵二叉排序树。



## 二叉排序树建立的思考题

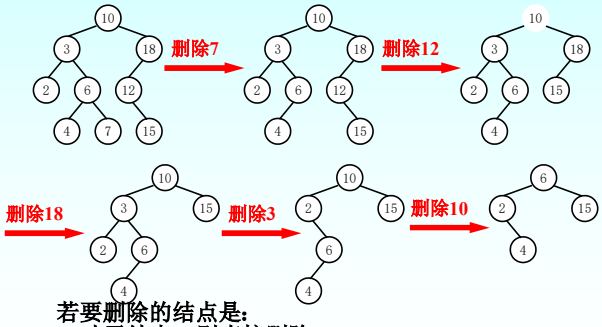
例：若将序列{10, 18, 3, 6, 12, 2, 4}中的“18”与“12”互换，试构成二叉排序树。

解：



结论：相同的数据元素，顺序不同，有可能构成不同的排序树。

## 二叉排序树的删除




若要删除的结点是：

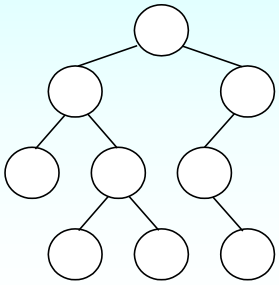
a.叶子结点，则直接删除

b.只有一个孩子，则用该孩子直接替代该结点

c.有两个孩子，则用该结点的左子树中最右边的结点代替之

## 二叉排序树的例题

例：有值分别为1，2，3，4，5，6，7，8，9的9个数据元素，如何安置在如下的二叉树结构中，使之形成一棵二叉排序树。



## 二叉排序树的查找

➢基本思想：二叉排序树本身可看成是一个有序结构，因此在二叉树上进行查找，与对分查找类似。

例：下述序列中，哪个可能是在二叉排序树上查找35时所比较过的关键字序列_____？

2，25，40，39，53，34，35

25，39，2，40，53，34，35

53，40，2，25，34，39，35

39，25，40，53，34，2，35

例：有数据{ 53，30，37，12，45，24，96}，从空二叉树开始逐个插入数据来形成二叉排序树，若希望高度最小，则应选择下面_____序列输入。

A. 45，24，53，12，37，96，30

B. 37，24，12，30，53，45，96

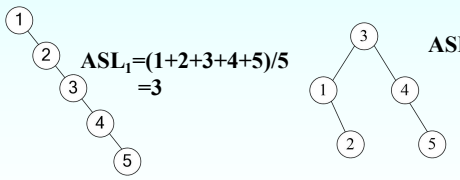
C. 12，24，30，37，45，53，96

D. 30，24，12，37，45，96，53

## 二叉排序树查找的算法分析

➢算法分析：其查找过程是走一条从根结点到所在结点的路径，与各个结点的比较次数＝路径长度加，则

$$ASL = \frac{1}{n}\left[\sum_{i=1}^{n}(\text{路径}+1)\right]$$




结论：1）二叉排序树的ASL与其本身的形态有关；

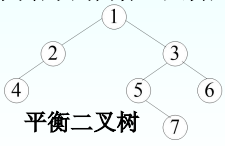
2）二叉排序树的形态比较均衡时查找效率很好，其形态均衡时查找能得到最好的效率，这就是平衡二叉排序树。

## 7.2.2 平衡二叉树（AVL树）

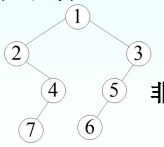
➢定义：平衡二叉树是前苏联数学家Ablelso-Velski和Landis发明的，或是一棵空树，或是具有下列性质的二叉树：它的左、右子树都是平衡二叉树，且左子树和右子树的深度之差的绝对值不超过1。

结点的平衡因子：结点的左子树深度减去其右子树深度，可能为－1，0和1。

例：判断下列两棵二叉树是否为平衡二叉树。




平衡二叉树




非平衡二叉树

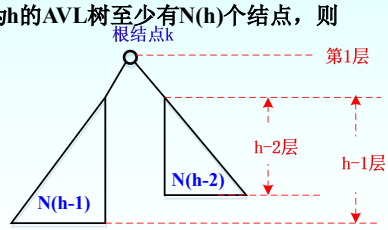
例：高度为5的平衡二叉树中，最少含有_____个结点，最多含有_____个结点。

## 平衡二叉树的最多和最少的结点数

➢ 深度为h的AVL树的最多结点是深度为h的满二叉树的结点数。

➢ 深度为h的AVL树至少有几个结点？

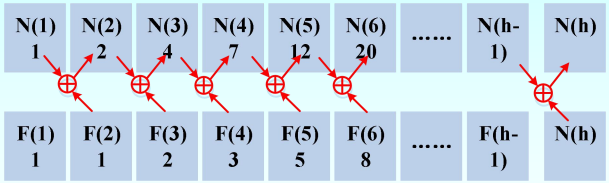
假设深度为h的AVL树至少有N(h)个结点，则




得， N(1)=1, N(2)=2, N(h)=N(h-1)+N(h-2)+1　　h>2

这个关系式和斐波那契序列（1, 1, 2, 3, 5, 8, 13, 21, 34…）即F(1)=1, F(2)=1, F(h)=F(h-1)+F(h-2)极为相似。

## 平衡二叉树的最少和最多结点数




两者之间的对应关系：　N(h)=N(h-1)+F(h)

与N(h)=N(h-1)+N(h-2)+1 比较，得　N(h)=F(h+2)-1

由于斐波那契序列满足渐进公式：$F(h)=\frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^{h}$

所以，得　$N(h)=\frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^{h+2}-1$

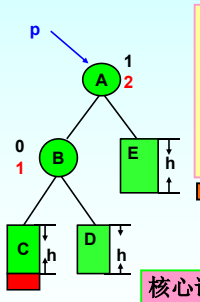
## 二叉排序树的平衡化旋转

➢一般的二叉排序树是不平衡的，可以通过平衡化旋转的方法使其既保持有序性，又具有平衡性。

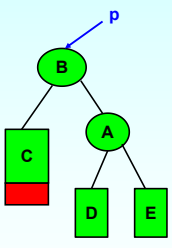
➢如果新插入的结点改变了原来的平衡性，则自插入点逆向找到第一个失衡结点，然后根据LL, RR, LR, RL四种不平衡形态进行调整。

➢只需要做一次旋转就可以达到平衡
——单旋转（LL、RR）
需要做两次旋转才可以达到平衡
——双旋转（LR、RL）

## LL型调整

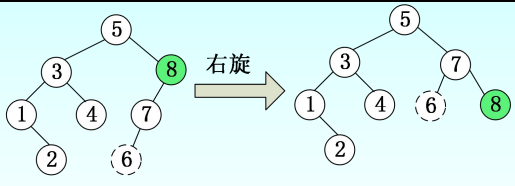



调整方法：
失衡结点A的左结点B单向右旋，成为根结点；A顺时针下拉成为B的右子树，而B的原右子树D成为A的左子树。

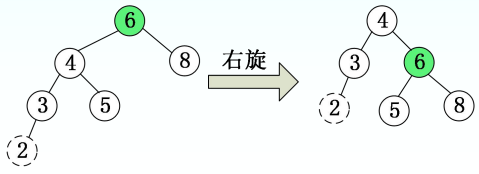



核心语句：
lc=p->lchild; //lc指向B
p->lchild=lc->rchild; //把B结点的右子树挂接为A的左子树
lc->rchild=p; //A结点成为B的右孩子
p=lc; //p指向新的根结点

## LL型调整的例题

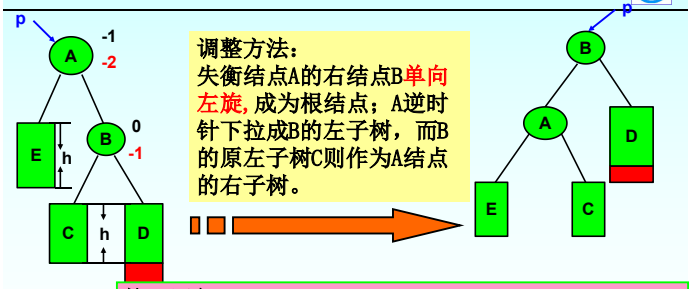



插入6后破坏了平衡——以7为根进行调整




插入2后破坏了平衡——以4为根进行调整

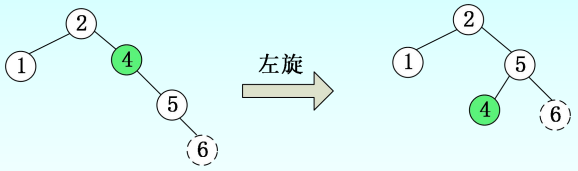
## RR型调整



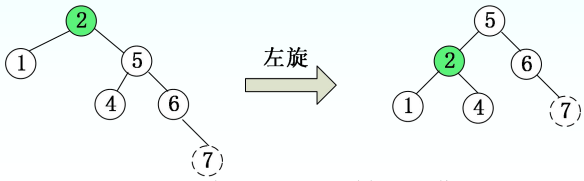

调整方法：
失衡结点A的右结点B单向左旋，成为根结点；A逆时针下拉成B的左子树，而B的原左子树C则作为A结点的右子树。

核心语句：
```
lc=p->rchild; //lc指向B结点
p->rchild=lc->lchild; //把B结点的左子树挂接为A的右子树
lc->lchild=p; //A结点成为B的左孩子
p=lc; //p指向新的根结点
```

## RR型调整的例题

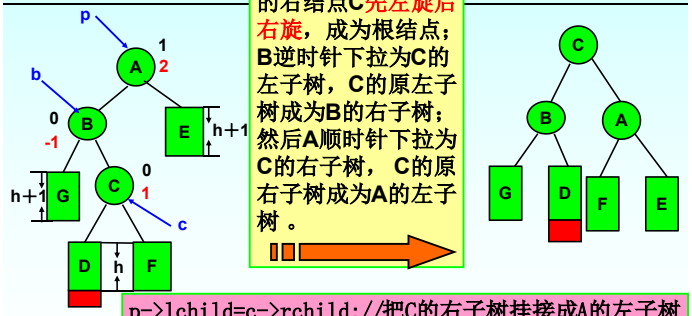



插入6后破坏了平衡——以5为根进行调整




插入7后破坏了平衡——以5为根进行调整

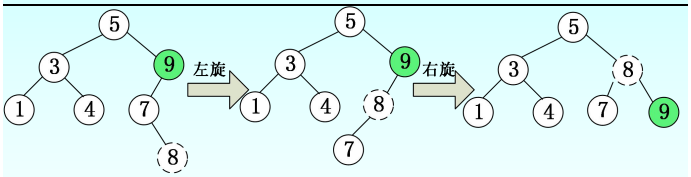
## LR型调整



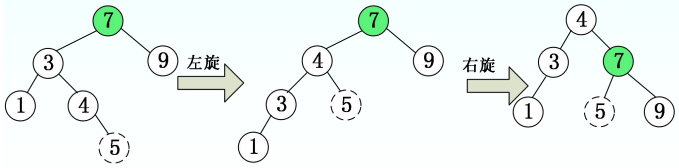

失衡结点A的左结点的右结点C先左旋后右旋，成为根结点；B逆时针下拉为C的左子树，C的原左子树成为B的右子树；然后A顺时针下拉为C的右子树，C的原右子树成为A的左子树。

```
p->lchild=c->rchild;//把C的右子树挂接成A的左子树
b->rchild=c->lchild;//把C的左子树挂接成B的右子树
c->lchild=b;//把B挂接成C的左子树
c->rchild=p;//把A挂接成C的右子树
```

## LR型调整的例题

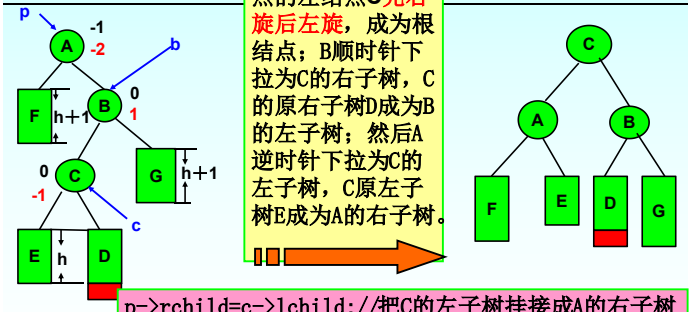



插入8后破坏了平衡——以8为根进行调整




插入5后破坏了平衡——以4为根进行调整

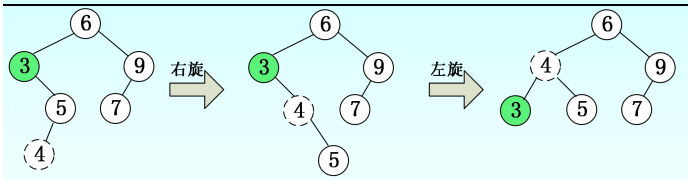
## RL型调整



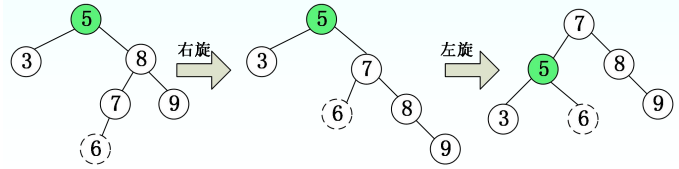

失衡结点A的右结点的左结点C先右旋后左旋，成为根结点；B顺时针下拉为C的右子树，C的原右子树D成为B的左子树；然后A逆时针下拉为C的左子树，C原左子树E成为A的右子树。

```
p->rchild=c->lchild;//把C的左子树挂接成A的右子树
b->lchild=c->rchild;//把C的右子树挂接成B的左子树
c-rchild=b;//把B挂接成C的右子树
c->lchild=p;//把A挂接成C的左子树
```

## RL型调整的例题



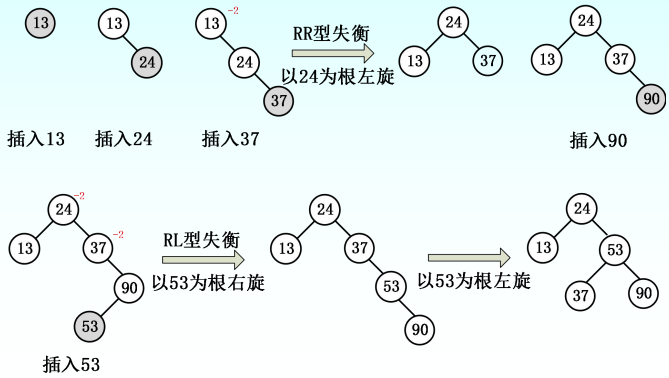

插入4后破坏了平衡——以4为根进行调整




插入6后破坏了平衡——以7为根进行调整

## AVL树生成的例题

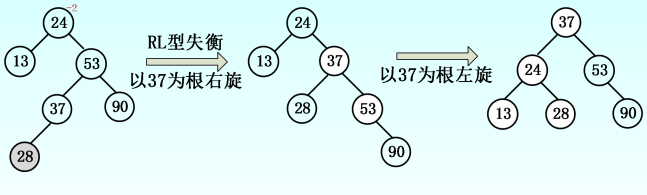
例：按关键字{13，24，37，90，53，28，98} 生成一棵AVL树。

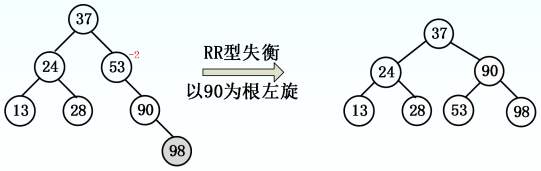






## AVL树生成的例题（续）




## AVL树查找的效率分析

在平衡二叉树上进行查找的过程和二叉排序树相同。因此，在查找过程中和关键字进行比较的次数不会超过树的深度。

思考题：

已知长度为12的表(Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec)。

(1) 试按表中元素的次序依次插入一棵初始为空的二叉排序树，请画出插入之后的二叉排序树，并求在等概率情况下查找成功的平均查找长度。

(2)若对表中元素先进行排序构成有序表，画出判定树，求在等概率的情况下对此表进行折半查找成功的平均查找长度。

(3)按表中元素顺序，构造一棵AVL树，画出平衡二叉树过程，并求其在等概率情况下查找成功的平均查找长度。

字母次序表：ABCDEFGHIJKLMNOPQRSTUVWXYZ

## 7.3 哈希表

7.3.1 哈希表概述

7.3.2 常用的哈希函数

7.3.3 冲突的解决方法

## 7.3.1 哈希表的概述

•哈希表基本思想：

Hash查找是通过对关键字进行某种运算，直接求出该关键字在某序列中的地址。

Hash表查找是关键字到地址的直接转换方法，而不需要反复比较。

•哈希函数—在关键字Key与序列地址之间建立某种函数关系。

d=H(key)　　d—Hash表地址

## 7.3.2 常用的哈希函数

•常用的哈希函数

1）直接定址法

2）数字分析法

3）平方取中法

4）折叠法

5）除留余数法※

## 1）直接定址法

- 取关键字或关键字的某个线性函数值为哈希地址

  d=H(key)=key 或 d= H(key)=a·key+b， a, b为常数
- 特点：地址不会产生冲突；
- 缺点：要求关键字分布的规律性强，每个关键字之间的间隔相等

例：解放后出生人口调查表，H(key)=key +(-1948)

| 地址 | 01 | 02 | 03 | … | 22 | … |
|---|---|---|---|---|---|---|
| 年份 | 1949 | 1950 | 1951 | … | 1970 | … |
| 人数 | … | … | … | … | 15000 | … |
| … | | | | | | |

## 2）数字分析法

•假设有一组关键字，每个关键字由n位数字组成，如 $k_1k_2……k_n$，从中提取数字分布比较均匀的若干位作为哈希地址。

d=H(key) =“key的若干位”

| 关键字 | 哈希地址 |
|---|---|
| 100011211 | 12 |
| 100011322 | 13 |
| 100011413 | 14 |
| 100011556 | 15 |
| 100011613 | 16 |
| 100011756 | 17 |
| 100011822 | 18 |
| 100011911 | 19 |

取第6位和第7位作为哈希地址。

## 3) 平方取中法

•取关键字平方的中间几位作为哈希地址，具体取多少位视实际情况而定。

d=H(key) =“key的平方的中间几位”

| 关键字 | （关键字）$^2$ | 哈希地址 |
|---|---|---|
| 421 | 177241 | 72 |
| 423 | 178929 | 89 |
| 436 | 190096 | 00 |
| 424 | 179776 | 97 |
| 580 | 336400 | 64 |
| 533 | 284089 | 40 |
| 567 | 321489 | 14 |
| 537 | 288369 | 83 |

取中间两位

## 4) 折叠法

- 将关键字分为等长的几段(除最后一段外)，再将各段相加作为哈希地址。
  - ♣ 移位折叠：将各段高位/低位靠齐后相加。
  - ♣ 边界折叠：将奇数字段或偶数字段倒排后相加。

例：

关键字值：12320324111220，分为5段：123,203,241,112,20

移位折叠：

```
  123
  203
  241
  112
   20
 ----
  879
（高位对齐）
```

边界折叠：

```
  123
  302
  241
  211
   20
 ----
  897
（低位对齐）
```

## 5）除留余数法

•用关键字key除以一个适合的不大于哈希表长度m的正整数p，所得余数作为哈希地址。

$$d = H(Key) = Key \bmod p$$

mod－整除取余；

$p$－不大于Hash表长$m$的正整数

•Hash表的装填系数

$$\alpha = \frac{n}{m}$$

$n$－关键字的个数；m－Hash表长

## 哈希表的建立

例：设哈希表的装填系数a=0.75，采用除留余数法构造14个记录 (13,29,01,23,44,55,20,84,27,68,11,10,79,14)的哈希表。

解： a=0.75，n=14，则哈希表表长m=n/a=19

Hash函数——$d_i$=H(Key)=Key mod p （p取17）

注意：Hash地址从0开始，0~m-1

| Hash地址： | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Hash表： | | 01 | | 20 | 55 | | 23 | | | | 44 | | 29 | 13 | | | 84 | | |

27 地址发生冲突！

冲突发生——某个Hash函数对2个不同的关键字得到相同的Hash地址

## 7.3.3 冲突的解决方法

1. 开放定址法

（1）线性探测再散列

（2）平方探测再散列

（3）伪随机探测再散列

2. 再哈希法

在发生地址冲突时，计算另一个哈希函数地址，直到冲突不再发生。

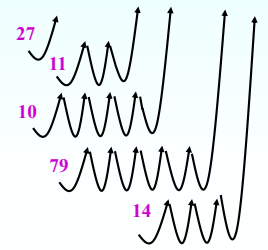
3. 链地址法

4. 建立一个公共溢出区

## （1）线性探测再散列

$d_{j+1}=(d_1+j) \bmod m \quad (j=1,2,\ldots\ldots)$

| Hash地址： | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Hash表： | 68 | 01 | | 20 | 55 | | 23 | | | | 44 | 27 | 29 | 13 | 11 | 10 | 84 | 79 | 14 |
| 比较次数： | 1 | 1 | | 1 | 1 | | 1 | | | | 1 | 2 | 1 | 1 | 4 | 6 | 1 | 7 | 5 |



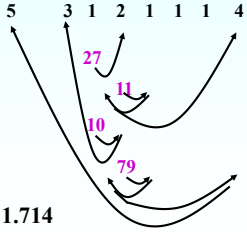

ASL = ( 1×9+2+4+6+7+5)/14 ≈ 2.357

## （2）平方探测再散列

$d_{2j}=(d_1+j^2) \bmod m \quad (j=1,2,\ldots\ldots)$

$d_{2j+1}=(d_1-j^2) \bmod m \quad (j=1,2,\ldots\ldots)$

| Hash地址： | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Hash表： | 68 | 01 | | 20 | 55 | | 23 | 79 | | 10 | 44 | 27 | 29 | 13 | 14 | 11 | 84 | | |
| 比较次数： | 1 | 1 | | 1 | 1 | | 1 | 5 | | 3 | 1 | 2 | 1 | 1 | 1 | 4 | 1 | | |




ASL = ( 1×10+5+3+2+4)/14 ≈ 1.714

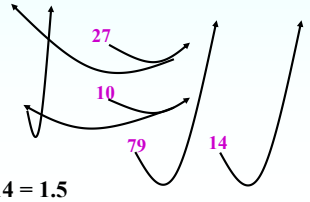
## （3）随机探测再散列

$d_{j+1}=(d_1+R_j) \bmod m \quad (j=1,2,\ldots\ldots)$

$R_j$是一组随机数列

取随机数序列为：3，16，55，44，...，得

| Hash地址： | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Hash表： | 68 | 01 | | 20 | 55 | | 23 | 27 | 10 | | 44 | 11 | 29 | 13 | 79 | | 84 | 14 | |
| 比较次数： | 1 | 1 | | 1 | 1 | | 1 | 3 | 4 | | 1 | 1 | 1 | 1 | 2 | | 1 | 2 | |




ASL = ( 1×10+3+4+2+2)/14 = 1.5

## 哈希表的查找

例：根据哈希函数$d_1$=Key mod 17、采用线性探测$d_{j+1}=(d_1+j) \bmod m$构建哈希表。

| Hash地址： | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Hash表： | 68 | 01 | | 20 | 55 | | 23 | | | | 44 | 27 | 29 | 13 | 11 | 10 | 84 | 79 | 14 |
| 比较次数： | 1 | 1 | | 1 | 1 | | 1 | | | | 1 | 2 | 1 | 1 | 4 | 6 | 1 | 7 | 5 |

（1）查找成功

——与关键字的比较次数有关

ASL = ( 1+1+1+1+1+1+2+1+1+4+6++1+7+5)/14 ≈ 2.357

（2）查找失败

——与该位置探测到第一个没有关键字的地址位置有关

ASL = ( 3+2+1+3+2+1+2+1+1+1+12+11+10+9+8+7+6+5+4)/19≈4.684

## 哈希表的查找分析

- 采用不同的解决冲突方法，平均查找长度也可能不同。
- 与线性查找、对分查找相比，ASL较小。
- 平均查找长度是装填系数a的函数。适当选择a的值是很关键的。

线性探测：$ASL=\frac{1}{2}\left(1+\frac{1}{1-a}\right)$

随机或平方探测：$ASL=-\frac{1}{a}\ln(1-a)$

与a有关，a越小，冲突越少，但浪费空间越大

## 链地址法

将哈希地址相同的记录存储在同一链表中，并用哈希表存放链表的头指针。

例：已知一组关键字(19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79)，哈希函数为：H(key)=key MOD 13，用链地址法处理冲突。

